McMaster University

W BOOTH SCHOOL OF ENGINEERING PRACTICE AND TECHNOLOGY

SEPT Interactive Laboratory Platform

Phase A — Developer Guide. How to run the platform on your own machine with nothing but Node, drive every part of it by hand, and satisfy yourself that it does what it claims — written for someone who has never started it before.

REPOSITORY
`Majd-214/SEPT-ILP`

BRANCH
`claude/phase-a-platform`

REQUIRES
`Node 20+ · no admin rights`

DOCUMENT DATE
18 August 2026

AUDIENCE

STATUS

Developers and maintainers	Phase A prototype — pre-pilot

What is in this guide

Sections 1–4 get the platform running and publish a site. Sections 5–9 walk through each feature as its real user would, and end with how labs actually reach students. Section 11 is the checklist that proves the whole thing works.

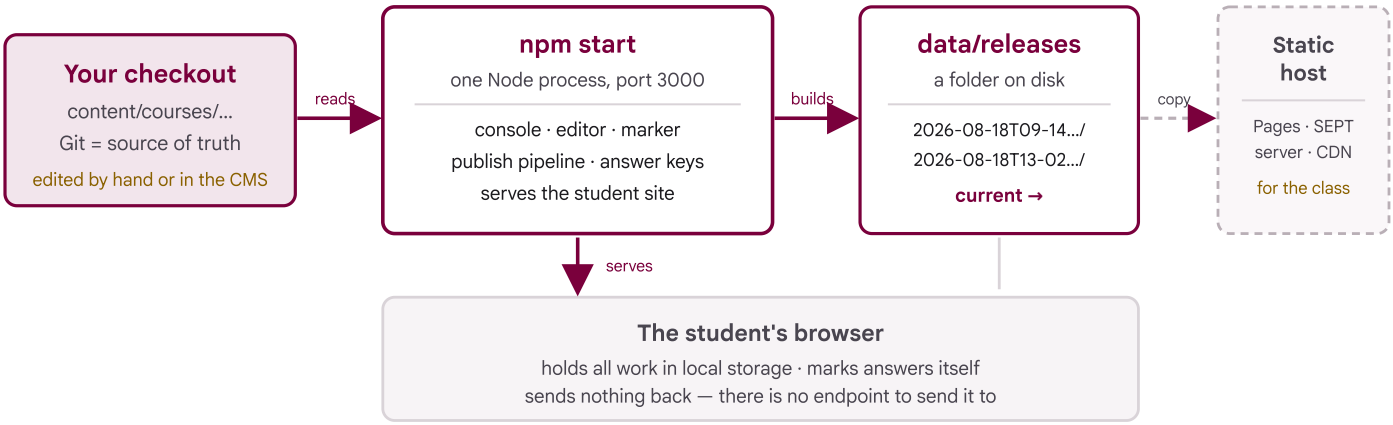
IF YOU ONLY READ ONE PAGE

`npm run setup` once, then `npm run seed` and `npm start`. Copy the sign-in link the terminal prints, open it, and press **Publish all courses**. That is the entire loop; everything after section 4 is detail. In WebStorm the same three steps are buttons — see section 2.

There is no container, no database server, and no mail server. That is a deliberate constraint, not a simplification for the guide: the machines this has to run on are School computers without virtualization enabled and without administrator rights. Every dependency the platform has is a JavaScript file in this repository.

0 The mental model

Your Git checkout is the truth. A build turns it into a frozen release. One process hands that release to whoever asks. Nothing flows backwards.



A gate failure stops the arrow at "builds". The live release never moves, so students keep the last good one.

Who touches what

Person	Where they go	What they can do
Student	the hosted lab link	Everything, with no account. Work lives in their browser and in files they download.
Instructor	`localhost:3000/login`	Their own courses: edit content, read answer keys, download exports, run the marker.
Admin	`localhost:3000/login`	All of the above, everywhere, plus inviting people and pressing Publish.

> **THE ONE RULE WORTH INTERNALISING**
>
> The server has no route that accepts student data. Not a disabled one — none exists, and a test asserts it. When you are wondering "where did that student's answer go?", the answer is always: into their browser, and into the file they downloaded.

1 Before you start

The list is short, and every item can be satisfied without administrator rights.

Requirement	How to check	Notes
Node 20 or newer	`node --version`	Node 22 LTS is a good default. It installs per-user on Windows and macOS; no admin rights and no virtualization needed. WebStorm can also point at a Node it manages.
WebStorm knows about Node	Settings → *Languages & Frameworks* → *Node.js*; the interpreter field should be filled in	Only needed for the run buttons in section 2; the terminal works regardless.
Port 3000 is free	`lsof -i :3000` (macOS/Linux) · `netstat -ano | findstr :3000` (Windows)	Or just use another: `PORT=3100 npm start`.
≈ 500 MB free disk	—	Dependencies and one Chromium for the accessibility gate. Each published release adds a few tens of MB.
Network for the first install ONCE	`npm run setup` completes	If a locked-down network blocks the Chromium download, everything else still works — see the callout below.

> **IF THE CHROMIUM DOWNLOAD IS BLOCKED**
>
> Only two things use it: the accessibility gate and the single-file export test. Build with `--skip-a11y` and the rest of the pipeline runs normally — but be honest in your notes that the gate was *skipped*, not *passed*. Run it later on a machine that can reach the download, before anything goes in front of students.

Get the branch and install

```
# in WebStorm: Git → Branches → claude/phase-a-platform → Checkout
git checkout claude/phase-a-platform
git pull

npm run setup   # all packages + a Chromium for the gates (a few minutes, once)
```

2 First run

Two commands. The repository also ships WebStorm run configurations, so you can do the whole thing without opening a terminal at all.

1. **Create the demo accounts.**

   ```
   npm run seed
   ```

   This creates an administrator and an instructor and issues each a one-time sign-in link. **Copy the admin link it prints** — you need it in a moment.

2. **Start the platform.**

   ```
   npm start
   SEPT-ILP platform listening on http://0.0.0.0:3000
   ```

   Leave it running. `Ctrl` + `C` stops it.

3. **Paste the sign-in link into a browser.**

   You are now signed in as an administrator, on the console dashboard.

The same thing, in WebStorm

The repository carries ready-made run configurations under `.run/`. They appear in the run-configuration dropdown at the top right the moment you open the project — no setup:

Configuration	Does
1 · Start the platform	`npm start` — the run window shows the log, including sign-in links.
2 · Seed accounts	`npm run seed` — creates or refreshes the demo accounts.
3 · Build (all gates)	`npm run gates` — the full pipeline on the pilot course, without publishing.
4 · All tests	`npm run test:all` — all four suites.

You can also click the small run arrows in the gutter of `package.json` beside any script, or use the *npm* tool window.

> **WHERE STATE LIVES**
>
> Everything the running platform creates goes in `data/` : the accounts database, published releases, editor drafts, and sent mail. It is git-ignored and disposable — delete the folder and re-run `npm run seed` to start clean. Content is never in there; content is in Git.

3 Signing in

There are no passwords anywhere in this system, and no mail server either. You ask for a link; the link is written to disk and printed in the terminal. When McMaster single sign-on arrives in Phase B it replaces exactly this one step and nothing else.

```
Sign-in link for admin@demo

http://localhost:3000/auth/SSZgXZPw5uQ36CHxdKuecPiQoOnj0_yWjPA9

Saved as /…/SEPT-ILP/data/mail/2026-08-18T13-34-44-621Z-admin-demo.eml
```

Every message is also kept as a readable `.eml` under `data/mail/`, so a link you scrolled past is never lost. Links are single-use and expire after 30 minutes; to get a fresh one, go to `/login` and enter the address.

> **THIS TRANSPORT IS FOR YOUR MACHINE ONLY**
>
> Anyone who can read the terminal or the `data/` folder can sign in as anybody. The platform therefore **refuses to start** using it when `BASE_URL` is not a localhost address, unless you opt in explicitly. A real deployment names a mail relay — see section 13.

What the console holds

Nav item	Who sees it	Purpose
Dashboard	everyone	Course list, the Publish button, and the log of every publish run.
Invites	admins	Add a colleague and choose which courses they can touch.
Link sheet	everyone	The canonical URL for each lab, with copy buttons and a suggested Avenue to Learn topic title.
Exports	everyone	Download single-file labs and course ZIPs from the live release.
Editor	everyone	The content CMS (section 6).
Marker	everyone	The auto-marker (section 7).

Try the role boundary

Sign in as `instructor@demo` in a private window and confirm the console is narrower: *Invites* is gone, and the Publish button is replaced by a line explaining that publishing is an administrator's job. Both are enforced server-side, not merely hidden.

4 Publishing

Publishing is the only way content reaches students, and it is a one-way door guarded by six checks. If any check fails, students keep seeing exactly what they saw before.

1. **On the dashboard (as admin), press Publish all courses.**
2. **The run appears in the table below, marked *running*. Open its report.**

   Give it two to six minutes: it builds both demo courses and runs a real browser over every page for the accessibility scan. Refresh the report to follow along.
3. **A finished run shows `OK` and a gate report like this:**

```
Building /…/content/courses/smrttech-3cc3 …
  gate inline-style     … pass
  gate class-allowlist  … pass
  gate answer-leak      … pass
  gate accessibility    … pass
  gate link-integrity   … pass

Build complete: data/releases/2026-08-18T13-02-44-903Z/smrttech-3cc3/site
```

What each gate refuses to let through

Gate	Blocks
schema	Content that does not match the JSON schemas — a quiz with no correct option, an image with no alternative text, a checkpoint with no way to complete it.
inline-style	Any `style=` attribute. All styling must come through the design system, so the whole platform can be restyled from one place.
class-allowlist	CSS classes the stylesheet does not define — the usual cause of a component that silently renders unstyled.
answer-leak	Any expected answer appearing in a student-facing page, including one split across markup or echoed onto the portal.
accessibility	WCAG 2.0 A/AA violations, scanned by axe-core in a real browser on every page.
link-integrity	Internal links and assets that do not resolve.

What a release looks like on disk

```
data/releases/2026-08-18T13-02-44-903Z/
└── smrttech-3cc3/
    ├── site/      ← students get this, and only this
    ├── keys/      ← answer keys; served only to signed-in staff
    └── bundles/   ← single-file exports and course ZIPs

data/current → data/releases/2026-08-18T13-02-44-903Z/   ← one symlink
```

Old releases are kept on purpose: rolling back is repointing that symlink, which is why the runbook's rollback procedure is three lines long.

> **NOW OPEN THE STUDENT SITE**
>
> `localhost:3000/c/smrttech-3cc3/lab-01/` — no login, no cookie. That short URL is the canonical one; it redirects to the full path, and it is the shape you publish to students (section 9). The *Link sheet* page has every one of them with a copy button.

5 Being a student

Spend ten minutes here as if you were in the lab. This is the part of the system everything else exists to serve, and most of its interesting behaviour is invisible from the console.

Open `localhost:3000/c/smrttech-3cc3/lab-01/` — ideally in a private window, so you see a genuinely fresh state.

- [ ] **Progression is locked.** Only the first checkpoint is open; later ones show a padlock in the left rail and their controls are disabled. Answer everything in checkpoint one, press *Complete checkpoint*, and watch the next unlock.
- [ ] **Going back un-completes things.** Return to a finished checkpoint and clear a required field. Its confirmation is revoked, and so is every checkpoint after it — completion is derived from the work, never a sticky flag.
- [ ] **The knowledge base never costs you your place.** Click any dotted knowledge link. A panel opens over the lab with the topic in it; close it and nothing has moved. That was the original design goal: reference material must never be a navigation away from work.
- [ ] **Work survives a reload.** Refresh the page mid-lab. Everything is still there — it is in the browser's local storage.
- [ ] **The progress file is the real record.** Open the *Progress* panel at the page edge, download the file, clear site data, then restore it. This is what a student does when they move between the lab computer and their laptop.
- [ ] **Finish the lab and download the submission package.** The button stays disabled until every checkpoint is confirmed and the identity fields are filled — hover it to see what is still missing.

Inside the submission ZIP

Entry	Contents
`completion.json`	Identity, every answer, checkpoint confirmation times, an auto-score, and a SHA-256 integrity hash of the record.
`evidence/…`	Every file the student attached, named after the evidence slot it belongs to.

PROVE THE PRIVACY CLAIM YOURSELF

Open DevTools → Network, then work through a checkpoint and download the package. You will see the page load and nothing else: no XHR, no beacon, no form post. The marking happens in the page against salted hashes, and the answer key it is checking against is not in the page at all — search the source for the expected value and you will not find it.

6 Editing content

The editor is a thin layer over Git. It never becomes a second source of truth: a draft lives outside the repository, and *Apply* turns it into an ordinary commit with your name on it.

1. **Console → Editor → SMRTTECH 3CC3 → *Edit* on Lab 1.**
   You get the lab as a list of blocks, grouped by checkpoint, each with move, edit and delete.
2. **Find a `quiz` block, press *Edit*, and reword its `prompt`.**
3. **Press Save draft.**
   The draft is validated immediately against the real content schemas. A clean draft says so.
4. **Press Apply to repository.**
   You get a commit hash back. In WebStorm, `Alt` + `9` → *Log* shows the commit, authored by the address you signed in with.
5. **Return to the dashboard and Publish, then reload the lab page to see your wording live.**

Now break it on purpose

Edit the same block and delete its `prompt` line entirely, then *Save draft*. The draft still saves — drafts are allowed to be wrong — but a red error appears pinned to that exact block, and *Apply* refuses. This is the same validator the build uses, so the editor and the pipeline can never disagree about what is valid.

Images

The upload box at the top of the editor requires alt text before it will accept anything, and it identifies files by their actual bytes rather than their name or declared type — a script-bearing SVG renamed `.png` is rejected. On success it stores the image and inserts a figure block.

> **COMMITS ARE LOCAL UNTIL YOU PUSH**
>
> *Apply* commits into your checkout. Push it yourself when you are happy: `Ctrl` + `Shift` + `K`. Nothing in the platform pushes on your behalf.

7 Marking submissions

The marker runs entirely in your browser. It re-derives every mark from the student's own recorded answers against the published key — it never trusts the score the student's page calculated.

1. **Console → Marker.**
   The keys table fills itself from the live release. If it is empty, you have not published yet.
2. **Drag the submission ZIP from section 5 onto the drop zone.**
   It matches the key by lab and content version, marks every item, and shows a total.
3. **Press Details.**
   Every item is listed with what the student answered and why it earned what it did.

What it can mark

Item type	How it is judged
choice	The recorded selection against the key's accepted options.
value	Numeric with a tolerance band (and an optional wider band for partial credit), or text with case and whitespace normalised.
formula	The key's expression evaluated *over that student's own measurements* — so a wrong resistance reading with an arithmetically correct calculation still earns the calculation marks.
evidence	The file must actually be present in the ZIP, not merely named in the record.
checkpoint	Confirmed, cross-checked against the recorded requirement counts.

See the formula behaviour for yourself

Unzip the submission, open `completion.json`, change `t3-r-dark` to a different resistance, re-zip, and drop it in again. The expected voltage moves with it: the breakdown will read *"answered 0.455, expected 0.45 from their own inputs — within tolerance"*. That is the difference between marking a lab and marking an answer sheet.

Advisory flags, never verdicts

The marker raises a flag — and only a flag — when something deserves a human look: a mismatched content version, an integrity hash that does not match, two submissions with identical answers under different names, a checkpoint confirmed with unmet requirements. It never withholds marks on suspicion. The judgement stays yours.

Exports

Button	Gives you
Brightspace CSV	One row per student number, in the format Avenue to Learn's Grades → Import accepts directly.
Full breakdown CSV	Every per-item mark and every review flag, for your records or a spreadsheet.
Feedback files	A ZIP of one Markdown file per submission, ready to return to students.

> **CONFIRM THE NO-EGRESS CLAIM**
>
> Open DevTools → Network before you drop a submission. The only requests you will ever see are the `/keys…` fetches at load. The page's Content-Security-Policy forbids every other destination, the server has nowhere to receive submissions, and a test in the suite fails the build if anyone adds a network call to this app.

8 Single-file exports

Every publish also produces each lab as one self-contained HTML file — stylesheet, fonts, scripts and images folded in — for when a lab has to live inside the LMS rather than behind a link.

1. **Console → Exports → download `smrttech-3cc3-lab-01-single.html`.**
2. **Open it straight from your downloads folder, with no server involved.**
   It is a complete lab: quizzes mark, checkpoints gate, the progress file downloads. Disconnect your network first if you want to be certain.

> **TEACH THE STORAGE CAVEAT, OR FIELD THE SUPPORT TICKETS**
>
> Browsers separate storage by origin. Work done in a single-file copy stays in that copy — it will not appear on the hosted site, and vice versa. The progress file is the bridge between them. If a course uses both forms, one sentence in the Avenue module text prevents nearly every confused email: *"your work saves in whichever copy you use; use the progress file to move it."*

The files are large — roughly 20–25 MB, dominated by embedded images — which is fine for an LMS file topic and poor for email. Prefer the canonical link whenever the choice is yours.

9 Getting labs to students

The platform on your machine is for staff. Students get plain static files from wherever the School hosts them, linked from Avenue to Learn — the same delivery model the original 3CC3 portal used, and the reason none of this needs a server at the far end.

Build the files

```
npm run build:all

dist/smrttech-3cc3/site/      ← upload this
dist/smrttech-3cc3/bundles/   ← single-file labs and course ZIPs
dist/smrttech-3cc3/keys/      ← NEVER upload this (instructor answer keys)
```

> **UPLOAD `site/` ONLY**
>
> `keys/` sits *outside* `site/` precisely so that copying the site folder cannot take the answer keys with it. Copy the `site` directory, never its parent.

Where to host it

Option	Operations	Notes
GitHub Pages	Already wired: every push to `main` builds and deploys	Simplest. Public at an unlisted URL — fine for lab manuals, which contain no answers and no student data.
Cloudflare Pages	Push-to-deploy from the repository	Adds a CDN, and Cloudflare Access can restrict to `@mcmaster.ca` if leadership wants the content gated.
SEPT server	One static vhost; deploy by copy or rsync from CI	Keeps everything on McMaster infrastructure; can sit behind campus SSO if UTS provisions it.
ZIP into A2L	Upload to Manage Files; re-upload to update	The always-works fallback. Inherits SSO for free, but runs inside Brightspace's iframe, where storage is partitioned — the progress file becomes a daily necessity rather than a safety net.

The recommendation

Host the site (Pages, Cloudflare, or the SEPT server) and put **links** in Avenue to Learn that open in a new tab. That single choice is the biggest reliability win available: storage stops being partitioned, so student work persists normally, and print, bookmarks, and multi-tab reference all behave. Access control stays honest either way — the links live behind A2L, the pages contain no answers and no way to submit anything, and real submissions still go through the A2L dropbox.

The console's **Link sheet** page gives you each lab's URL and a suggested topic title, with copy buttons. Set `BASE_URL` to your hosting origin before publishing so the sheet prints the real links rather than `localhost`.

Full comparison, including the LTI path for Phase B, is in `docs/deployment.md`.

10 Running the tests

Eighty-seven tests across four suites, behind one command.

```
# once
npm run setup

# thereafter
npm run test:all
```

In WebStorm, use the **4 · All tests** run configuration, the run arrows in the gutter of `package.json`, or the *npm* tool window.

What each suite is guarding

Suite	Tests	Protects
`npm test` (renderer)	29	The hashing scheme, tolerance edges, key generation, the answer-leak gate, and single-file export parity against the hosted page.
`npm run test:platform`	11	Magic links, sessions, roles, CSRF, rate limiting, path-traversal defences, the absence of student routes, mail delivery with no mail server, and an accessibility scan of every console page.
`npm run test:marker`	41	Every marking rule, the formula evaluator's safety, ZIP reading, CSV formatting, and the no-egress guarantee.
`npm run test:admin`	6	Block skeletons stay schema-valid, the draft→validate→apply flow, course access, and upload hardening.

Running the gates directly

To exercise the full build without the console, run `npm run gates` (the pilot course in strict mode) or `npm run build:all` (both courses, plus single-file exports). This is what CI runs on every push, and it is the fastest way to check a content change before publishing.

> **EXPECT THE SQLITE NOTICE**
>
> On Node 22 you will see *"SQLite is an experimental feature"*. That is Node talking about its own built-in database module, not a problem with the platform, and it disappears on newer Node. The npm scripts pick the right flag for whichever version you have — which is why `npm test` works where a bare `node --test` may not.

11 Proving it works

This is the acceptance script the platform was built against, written out so you can walk it by hand. Roughly twenty minutes. If all eight boxes tick, every claim in this guide holds on your machine.

- [ ] **1 · The platform comes up seeded.** `npm run seed` prints two sign-in links and writes two files under `data/mail/`; `npm start` reports it is listening.
- [ ] **2 · An invitation reaches a new person.** Sign in as admin. Invites → invite `tester@demo` as an instructor for SMRTTECH 3CC3. Their link appears in the terminal; open it in a private window and you are signed in as them, with a narrower console.
- [ ] **3 · An edit reaches students only through the gates.** As that instructor, reword a Lab 1 quiz prompt in the editor → Save draft → Apply. Publish as admin. The report shows every gate passing, and `localhost:3000/c/smrttech-3cc3/lab-01/` shows the new wording.
- [ ] **4 · A student completes the lab with no account.** In a fresh private window, work through Lab 1. Put one measurement deliberately near the edge of its tolerance. Fill the identity fields, attach a file to each evidence slot, download the submission package.
- [ ] **5 · The marker marks it.** Back as the instructor, Marker → drop that ZIP. The key matches automatically, the formula items are judged against that student's own measurements, and the Brightspace CSV downloads with one row per student number.
- [ ] **6 · Answer keys stay inside their course.** As an instructor invited to only one course, request another course's key with a traversal: `/keys/smrttech-3cc3/..%2f..%2f..%2fdemo-101%2fkeys%2fdemo-101%2flab-01.key.json`. It must refuse — `400`, and no key contents.
- [ ] **7 · A single-file export works with no server.** Exports → download the Lab 1 single file → open from disk, offline. Quizzes still mark.
- [ ] **8 · The answer-leak gate really blocks a leak.** Add the text `4.68` to the footer in `renderer/src/pages/Page.js`, then run `node renderer/src/cli.js build content/courses/demo-101 --out /tmp/leak --skip-a11y`. The build must fail on `answer-leak`. Undo the edit and it passes again.

> **WHAT THIS ESTABLISHES**
>
> Content only reaches students through the gates; students need no account and leave no data on the server; marking is reproducible from what the student actually recorded; and answers cannot leak, either into a page or across a course boundary. Those are the four claims the pilot rests on.

12 When something breaks

Symptoms you are most likely to hit first, and what each one actually means.

Symptom	Cause	Fix
`ERR_UNKNOWN_BUILTIN_MODULE: node:sqlite`	A bare `node …` command on Node 22, which needs a flag for its built-in database.	Use the npm scripts (`npm start` , `npm run seed` , `npm run test:platform`) — they detect and pass it.
`EADDRINUSE :3000`	Something else has the port — often an earlier copy of the platform still running.	Stop it, or `PORT=3100 npm start` with `BASE_URL=http://localhost:3100` .
No sign-in link anywhere	The address has no account, or you are looking at the wrong terminal.	Check `data/mail/` . Unknown addresses are ignored silently, on purpose — the platform will not confirm who has an account.
Student site says *Nothing published yet*	Accurate — no release exists.	Sign in as admin and press Publish.
Publish fails on **accessibility**, browser missing	Chromium was never downloaded, or the install was blocked.	`npx playwright install chromium` . If the network forbids it, build with `--skip-a11y` and record the gate as owed.
Publish fails on **answer-leak**	Not a bug — a marked answer is printed in a student page.	The report names the file and the exact value. Fix the content. Never relax the gate.
Publish fails on **schema** right after an edit	Content edited outside the editor, bypassing validation.	Run `npm run validate` to see the same errors locally, with paths.
The platform refuses to start, naming `COOKIE_SECRET` or `SMTP_HOST`	Working as intended: `BASE_URL` is not localhost, so development defaults are refused.	Set both (section 13), or keep `BASE_URL` local while developing.
Everything is confusing; you want a clean slate	Accumulated local state.	Stop the platform, delete `data/` , run `npm run seed` . Content is in Git and is untouched.
Disk filling up	Releases accumulate by design (they are the rollback history).	`ls -dt data/releases/* | tail -n +6 | xargs rm -rf`

13 Reference

URLs

URL	What
`localhost:3000/c/<course>/<lab>/`	Canonical student lab link — the shape you publish.
`localhost:3000/login`	Faculty sign-in.
`localhost:3000/admin`	Console dashboard, publishing, publish history.
`localhost:3000/admin/links`	Link sheet with copy buttons and suggested A2L titles.
`localhost:3000/admin/exports`	Single-file labs and course ZIPs.
`localhost:3000/admin/editor`	Content editor.
`localhost:3000/marker/`	Instructor auto-marker.
`localhost:3000/keys/<course>`	Answer keys as JSON — role-gated.
`localhost:3000/healthz`	Liveness plus the current release path.

Environment variables

Variable	Default	Purpose
`PORT`	`3000`	Listening port.
`BASE_URL`	`http://localhost:3000`	The origin written into sign-in links and the link sheet. Set it to your hosting origin before publishing for students.
`COOKIE_SECRET`	dev value	Signs sessions and CSRF tokens. **Required for any non-localhost `BASE_URL`.** Generate with `openssl rand -hex 32`.
`SMTP_HOST` / `SMTP_PORT`	— / `25`	A mail relay. Setting the host switches mail from the local file sink to real delivery, which a non-localhost `BASE_URL` requires.
`MAIL_TRANSPORT`	`file`	`file` writes `.eml` files and prints links; `smtp` relays. Set explicitly only to override the automatic choice.
`MAIL_DIR`	`data/mail`	Where the file transport writes.
`SEED_ADMIN` / `SEED_INSTRUCTOR`	`admin@demo` / `instructor@demo`	Accounts `npm run seed` creates.
`DATA_DIR`	`data/`	Database, releases, drafts, and mail.
`PUBLISH_GIT_PULL`	off	Set to `1` to `git pull --ff-only` before each publish.
`PUBLISH_SKIP_A11Y`	off	Skips the accessibility gate. For machines with no browser only — never for a real publish.

Where things live

Path	Contents
`content/courses/`	The courses. This is the durable asset; everything else is machinery.
`schema/v1/`	JSON schemas for labs, blocks, knowledge, progress, answer keys.
`design-system/`	The one stylesheet and the browser runtime, in layers.
`renderer/`	Content → static site, plus every quality gate.
`apps/platform/`	The faculty service: auth, publishing, keys, exports.
`apps/admin/`	The content editor mounted by the platform.
`apps/marker/`	The client-side auto-marker.
`scripts/`	`start.mjs` and `seed.mjs` — what the npm scripts run.
`.run/`	WebStorm run configurations, shared through the repository.
`integrations/`	Phase B seams — interfaces only, nothing wired up.
`docs/`	runbook · architecture · marking · deployment · decisions/adr-001-cms
`data/`	Local state: database, releases, drafts, mail. Git-ignored and disposable.

The five invariants, and what enforces each

Invariant	Enforcement
No student data on the server	No student route exists; a test asserts every plausible submission endpoint returns 404/405.
No plaintext answers in public artifacts	The answer-leak gate, on every build, structurally and by text — including answers split across markup.
Git is the source of truth; builds are deterministic	The editor commits rather than storing; the build timestamp comes from the content commit, verified by byte-identical rebuilds.
All styling through the design system	The inline-style and class-allowlist gates; the console uses the same stylesheet and passes the same accessibility scan.
One clearly marked authentication seam	A single `OIDC SEAM` block in `apps/platform/src/auth.js`; roles and course membership stay local either way.

14 Honest status

What you are running is a complete Phase A prototype, verified end to end — not a finished production service. The distinction matters most in these places.

Solid VERIFIED

The rendering pipeline and its gates; the marking model and the marker; content editing through Git; publishing with atomic rollback; the student experience including gating, progress files and submission packages. All covered by tests and by the section 11 walkthrough.

Stubbed BY DESIGN

Authentication is email magic links, not MacID — deliberate, and confined to one seam. Grade return is a CSV you import by hand. Both are Phase B swaps, and their contracts already exist under `integrations/`.

Rough edges

Publishing rebuilds every course, not just the changed one. The editor edits blocks as JSON rather than generated forms (the reasoning is in ADR-001). Rollback is a documented command, not a button.

Decide before the pilot

Where the student site is hosted and who deploys it; who holds the admin account; whether single-file exports are used at all, given the storage caveat; and how student evidence files are retained once they reach Avenue to Learn.

If you do only three things before students see it

1. Pick the hosting (section 9) and publish the pilot course there, with `BASE_URL` set to that origin.
2. Walk section 11 against the hosted site, not just `localhost`.
3. Put the lab links in the Avenue to Learn module — set to open in a new tab — and add the one sentence about the progress file.

WHERE TO GO DEEPER

`docs/runbook.md` for day-to-day operations and rollback · `docs/architecture.md` for how the layers fit · `docs/marking.md` for the hashing scheme and its honest security claims · `docs/deployment.md` for the full hosting comparison · `docs/decisions/adr-001-cms.md` for why the CMS is what it is.